

COMPUTER ARCHITECTURE

A Quantitative Approach (5th Edition)

UNIT 1 – Fundamentals of Quantitative Design and Analysis & Instruction-Level Parallelism

Structured Exam-Revision Notes

Based on: Hennessy & Patterson, Chapter 1 slides (+ ILP / Pipelining material)

Contents

- Part 1 – Introduction & Classes of Parallelism
- Part 2 – Trends in Technology, Power, Cost & Dependability
- Part 3 – Measuring Performance & Principles of Computer Design
- Part 4 – Pipelining (Hazards, Forwarding, Branch Handling)
- Part 5 – Instruction-Level Parallelism (Dependences, Loop Unrolling, Strip Mining)
- Part 6 – Branch Prediction, Dynamic Scheduling, Tomasulo, Speculation
- Part 7 – Multiple Issue, VLIW & Advanced Instruction Delivery
- Quick Revision – Formula Sheet & Comparison Tables

PART 1 – Introduction & Classes of Parallelism

1.1 Computer Technology

- **Performance improvements** come from:
 - Improvements in **semiconductor technology** – feature size, clock speed
 - Improvements in **computer architectures** – enabled by HLL compilers, UNIX; led to **RISC architectures**
- Together these have enabled **lightweight computers** and **productivity-based managed/interpreted programming languages**.

1.2 Single-Processor Performance & Current Trends

- Performance growth of single processors was driven by RISC; then there was a **move to multi-processor**.
- Cannot continue to leverage **Instruction-Level Parallelism (ILP)** – **single-processor performance improvement ended in 2003**.
- New models for performance:
 - **Data-Level Parallelism (DLP)**
 - **Thread-Level Parallelism (TLP)**
 - **Request-Level Parallelism (RLP)**
- These require **explicit restructuring of the application**.

1.3 Classes of Parallelism

Classes of parallelism in applications	Classes of architectural parallelism
Data-Level Parallelism (DLP) Task-Level Parallelism (TLP)	Instruction-Level Parallelism (ILP) Vector architectures / GPUs Thread-Level Parallelism Request-Level Parallelism

How hardware exploits parallelism

1. **Instruction-Level Parallelism** exploits DLP, e.g. **pipelining** and **speculative execution**.
2. **Vector Architectures and GPUs** exploit DLP by applying a **single instruction to a collection of data in parallel**.
3. **Thread-Level Parallelism** exploits either DLP or TLP in a **tightly coupled hardware model** that allows interaction among parallel threads.
4. **Request-Level Parallelism** exploits parallelism among **largely decoupled tasks** specified by the programmer or the operating system.

1.4 Flynn's Taxonomy

Class	Full form	Notes / Examples
SISD	Single instruction stream, single data stream	Conventional uniprocessor
SIMD	Single instruction stream, multiple data streams	Vector architectures, multimedia extensions, graphics processor units (GPUs)

Class	Full form	Notes / Examples
MISD	Multiple instruction streams, single data stream	No commercial implementation
MIMD	Multiple instruction streams, multiple data streams	Tightly-coupled MIMD; Loosely-coupled MIMD: Cluster computing

MNEMONIC

Single/**M**ultiple Instruction × Single/**M**ultiple Data → SISR, SIMD, MISD, MIMD (first letter = instruction stream, second = data stream).

1.5 Defining Computer Architecture

"Old" view	"Real" computer architecture
Instruction Set Architecture (ISA) design, i.e. decisions regarding: registers, memory addressing, addressing modes, instruction operands, available operations, control flow instructions, instruction encoding	Specific requirements of the target machine. Design to maximize performance within constraints: cost, power, and availability. Includes ISA, microarchitecture, hardware.

SECTION SUMMARY

Parallelism (DLP, TLP, ILP, RLP) replaced raw clock-speed scaling after 2003. Flynn classifies machines as SISR/SIMD/MISD/MIMD. Real architecture = ISA + microarchitecture + hardware, under cost/power/availability constraints.

PART 2 – Trends in Technology, Power, Cost & Dependability

2.1 Technology

- If an ISA is to be successful, it must be designed to **survive rapid changes in computer technology**. A successful ISA may last decades – e.g. the core of the **IBM mainframe has been in use for nearly 50 years**.
- An architect must plan for technology changes that can increase the lifetime of a successful computer.
- To plan for the evolution of a computer, the designer must be aware of rapid changes in implementation technology. **Five implementation technologies** change at a dramatic pace and are critical to modern implementations.

2.2 Trends in Technology (Growth Rates)

Technology	Trend
Integrated circuit – transistor density	35%/year
Integrated circuit – die size	10–20%/year
Integrated circuit – integration overall	40–55%/year
DRAM capacity	25–40%/year (slowing)
Flash capacity	50–60%/year ; 15–20X cheaper/bit than DRAM
Magnetic disk technology	40%/year ; 15–25X cheaper/bit than Flash; 300–500X cheaper/bit than DRAM

2.3 Bandwidth and Latency

Term	Meaning	Improvement observed
Bandwidth (throughput)	Total work done in a given time	10,000–25,000X for processors; 300–1200X for memory and disks
Latency (response time)	Time between start and completion of an event	30–80X for processors; 6–8X for memory and disks

KEY POINT

Bandwidth has improved **far more** than latency (“bandwidth improves at least as the square of latency” trend, shown in the log-log plot of milestones).

2.4 Transistors and Wires

- **Feature size** = minimum size of transistor or wire in x or y dimension. It went from **10 microns in 1971 to 0.032 microns in 2011**.
- Transistor performance scales **linearly**.
- **Wire delay does not improve with feature size!**
- Integration density scales **quadratically**.

2.5 Power and Energy

- Problem: **Get power in, get power out.**

DEFINITION

Thermal Design Power (TDP) – characterizes **sustained power consumption**; used as the target for the power supply and cooling system. It is **lower than peak power, higher than average power** consumption.

- Clock rate can be reduced dynamically to limit power consumption.
- **Energy per task** is often a better measurement than power.

Dynamic Energy and Power

- **Dynamic energy** – energy when a transistor switches from 0→1 or 1→0:

$$\text{Energy(dynamic)} = \frac{1}{2} \times \text{Capacitive load} \times \text{Voltage}^2$$

- **Dynamic power:**

$$\text{Power(dynamic)} = \frac{1}{2} \times \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency switched}$$

- **Reducing clock rate reduces power, not energy.**

EXAMPLE

Voltage/frequency: Processor with adjustable voltage; a **15% reduction in voltage** also gives a 15% reduction in frequency. Capacitance is unchanged, so:

Energy(new)/Energy(old) = $(0.85)^2 = 0.72$ → energy falls to about **72%** of the original.

Power(new)/Power(old) = $0.72 \times 0.85 = 0.61$ → power falls to about **61%** of the original.

EXAMPLE

Power vs energy: Processor A may have a **20% higher average power** than B, but if A executes the task in only **70% of the time** needed by B, its energy = $1.2 \times 0.7 = 0.84$ – clearly better.

- **Intel 80386** consumed ~**2 W**; a **3.3 GHz Intel Core i7** consumes **130 W**.
- Heat must be dissipated from a **1.5 × 1.5 cm chip** – this is the **limit of what can be cooled by air**.

Techniques for Reducing Power

1. **Do nothing well** – most microprocessors turn off the clock of inactive modules to save energy and dynamic power. E.g. if no floating-point instructions are executing, the clock of the FP unit is disabled.
2. **Dynamic Voltage-Frequency Scaling (DVFS)** – in periods of low activity there is no need to operate at the highest clock frequency and voltage. Modern microprocessors typically offer a few clock frequencies and voltages that use lower power and energy.
3. **Low-power state for DRAM, disks** – low-power modes to save energy.
4. **Overclocking / turning off cores (Turbo mode)** – if temperature rises beyond a certain threshold, turn off a few cores; performance varies with respect to time.

Static Power

$$\text{Power(static)} = \text{Current(static)} \times \text{Voltage}$$

- Scales with the **number of transistors**.
- To reduce: **power gating** – an IC design technique that reduces power consumption by **shutting off the current to blocks of the circuit that are not in use**.

	Dynamic power	Static power
Cause	Transistor switching	Leakage current (even when idle)

	Dynamic power	Static power
Formula	$\frac{1}{2} \times C \times V^2 \times f$	$I(\text{static}) \times V$
Reduced by	Clock gating, DVFS, lower clock/voltage	Power gating

2.6 Trends in Cost

- Cost is **driven down by the learning curve** (yield).
- **DRAM**: price closely tracks cost.
- **Microprocessors**: price depends on volume – **10% less for each doubling of volume**.

Integrated Circuit Cost

Cost of IC = (Cost of die + Cost of testing die + Cost of packaging and final test) / Final test yield

Cost of die = Cost of wafer / (Dies per wafer × Die yield)

Dies per wafer = $\pi \times (\text{Wafer diameter}/2)^2 / \text{Die area}$ – $\pi \times \text{Wafer diameter} / \sqrt{(2 \times \text{Die area})}$

- **Bose–Einstein formula:**

Die yield = $\text{Wafer yield} \times 1 / (1 + \text{Defects per unit area} \times \text{Die area})^N$

- Defects per unit area = **0.016–0.057 defects per cm²** (2010).
- **N** = process-complexity factor = **11.5–15.5** (40 nm, 2010).

EXAMPLE

Die yield: dies 1.5 cm on a side and 1.0 cm on a side; defect density = 0.031/cm², N = 13.5.

Die areas = 2.25 cm² and 1.00 cm².

Larger die: yield = $1/(1 + 0.031 \times 2.25)^{13.5} = \mathbf{0.40}$

Smaller die: yield = $1/(1 + 0.031 \times 1.00)^{13.5} = \mathbf{0.66}$

So less than half of the large dies are good, but two-thirds of the small dies are good.

EXAMPLE

Dies per wafer (300 mm wafer):

Die area 2.25 cm²: $\pi(30/2)^2/2.25 - \pi \times 30/\sqrt{2 \times 2.25} = 706.9/2.25 - 94.2/1.12 = \mathbf{270}$

Die area 1.00 cm²: $706.9/1.00 - 94.2/1.41 = \mathbf{640}$

The larger die is 2.25 times bigger, so there are roughly 2.25 times as many smaller dies per wafer.

2.7 Dependability

Module reliability

Term	Meaning / Formula
MTTF	Mean Time To Failure
MTTR	Mean Time To Repair
MTBF	Mean Time Between Failures = MTTF + MTTR
Availability	MTTF / MTBF

EXAMPLE

Disk subsystem MTTF: 10 disks (each 1,000,000-hour MTTF), 1 ATA controller (500,000-hour), 1 power supply (200,000-hour), 1 fan (200,000-hour), 1 ATA cable (1,000,000-hour). Assume lifetimes are **exponentially distributed** and failures are **independent**.

Failure rate(system) = $10 \times 1/1,000,000 + 1/500,000 + 1/200,000 + 1/200,000 + 1/1,000,000$

= $(10 + 2 + 5 + 5 + 1) / 1,000,000$ hours = $23 / 1,000,000$ hours = **23,000 FIT**

MTTF(system) = $1 / \text{Failure rate} = 1,000,000,000 / 23,000 \approx \mathbf{43,500 \text{ hours}}$ (just under 5 years).

KEY POINT

For independent components, **failure rates add**; MTTF of the system is the **inverse of the sum of failure rates**.

SECTION SUMMARY

Technology: density grows ~35%/yr, bandwidth outpaces latency, wires do not scale. Power:

$P(\text{dyn}) = \frac{1}{2} \cdot C \cdot V^2 \cdot f$; reduce via clock gating, DVFS, power gating. Cost: die yield falls sharply with die area.

Dependability: Availability = $\text{MTTF} / (\text{MTTF} + \text{MTTR})$.

PART 3 – Measuring Performance & Principles of Computer Design

3.1 Measuring Performance

- Typical metrics: **Response time** and **Throughput**.
- **Speedup of X relative to Y:**

$$\text{Speedup} = \text{Execution time}(Y) / \text{Execution time}(X)$$

Execution time

Type	Meaning
Wall-clock time	Includes all system overheads
CPU time	Only computation time

Benchmarks

Type	Example
Kernels	Matrix multiply
Toy programs	Sorting
Synthetic benchmarks	Dhrystone
Benchmark suites	SPEC06fp, TPC-C

3.2 Principles of Computer Design

Principle 1: Take Advantage of Parallelism

- E.g. multiple processors, disks, memory banks, **pipelining**, multiple functional units.

Principle 2: Principle of Locality

- **Reuse of data and instructions.**
- **Temporal locality** – recently accessed items are likely to be accessed in the near future.
- **Spatial locality** – items whose addresses are near one another tend to be referenced close together in time.

Principle 3: Focus on the Common Case – Amdahl's Law

DEFINITION

Speedup = time a program takes to execute in serial (with one processor) / time it takes to execute in parallel (with many processors).

$$S = T(1) / T(j) \quad \text{where } T(j) = \text{time to execute the program using } j \text{ processors}$$

- If there are **N workers** on a project, assume they can do a job in **1/N** time of one worker alone.
- If the strictly **serial part** takes **B×T(1)**, then the strictly **parallel part** takes **((1-B)×T(1))/N**. Substituting gives:

$$S = N / (B \times N + (1 - B))$$

- **General form (enhancement):**

$$\text{Execution time(new)} = \text{Execution time(old)} \times [(1 - \text{Fraction(enhanced)}) + \text{Fraction(enhanced)}/\text{Speedup(enhanced)}]$$

$$\text{Speedup(overall)} = \text{Execution time(old)} / \text{Execution time(new)} = 1 / [(1 - \text{Fraction(enhanced)}) + \text{Fraction(enhanced)}/\text{Speedup(enhanced)}]$$

EXAMPLE

Web serving: New processor is 10× faster on computation. Original is busy with computation **40%** of the time and waiting for I/O **60%**.

Fraction(enhanced) = 0.4; Speedup(enhanced) = 10

Speedup(overall) = 1 / (0.6 + 0.4/10) = 1/0.64 = **1.56**

EXAMPLE

FP square root (FPSQR) vs all FP instructions: FPSQR = 20% of execution time; FP instructions = 50% of execution time.

Alternative 1 – speed up FPSQR by 10: Speedup = 1/((1-0.2) + 0.2/10) = 1/0.82 = **1.22**

Alternative 2 – speed up all FP by 1.6: Speedup = 1/((1-0.5) + 0.5/1.6) = 1/0.8125 = **1.23**

Improving all FP operations is slightly better overall because of the higher frequency of FP operations.

KEY POINT

Amdahl's Law: overall speedup is **limited by the fraction of time that cannot be enhanced**. Make the **common case fast**.

The Processor Performance Equation

$$\text{CPU time} = \text{CPU clock cycles for a program} \times \text{Clock cycle time}$$

$$\text{CPU time} = \text{CPU clock cycles for a program} / \text{Clock rate}$$

$$\text{CPI} = \text{CPU clock cycles for a program} / \text{Instruction count}$$

$$\begin{aligned} \text{CPU time} &= \text{Instruction count} \times \text{Cycles per instruction} \times \text{Clock cycle time} \\ (\text{Instructions/Program}) \times (\text{Clock cycles/Instruction}) \times (\text{Seconds/Clock cycle}) &= \\ \text{Seconds/Program} &= \text{CPU time} \end{aligned}$$

Different instruction types having different CPIs

$$\text{CPU clock cycles} = \sum (\text{IC}_i \times \text{CPI}_i) \text{ for } i = 1 \text{ to } n$$

$$\text{CPU time} = [\sum (\text{IC}_i \times \text{CPI}_i)] \times \text{Clock cycle time}$$

Factor	Affected by
Clock cycle time	Hardware technology and organization
CPI	Organization and instruction set architecture
Instruction count	Instruction set architecture and compiler technology

SECTION SUMMARY

Performance is measured by response time & throughput; speedup = Time(Y)/Time(X). Design principles: parallelism, locality, common case (Amdahl). CPU time = IC × CPI × Clock cycle time.

PART 4 – Pipelining

4.1 Pipelining Basics

- Analogy: **pipelines** transport liquids or gases over long distances (1000s of km), built with periodic pump/compressor stations to keep the fluid flowing (e.g. refinery → city, 1000 km).
- **Inside the processor** the five stages are: **IF** (Instruction Fetch), **ID** (Instruction Decode), **EX** (Execution), **MEM** (Memory), **WB** (Write Back).
- **Processor pipelining**: instructions overlap in time (i1, i2, i3, i4 start one clock cycle apart).
- **Execution time of each instruction is still 5 cycles, but throughput is now 1 instruction per cycle.**
- **Initial pipeline fill time = 4 cycles**, after which 1 instruction completes every cycle.
- MIPS-1 instructions take **3, 4 or 5 cycles** (e.g. LW R1,0(R2) and ADD R3,R1,R2).

Some terminology

- **Pipeline stages**: IF, ID, EX, MEM, WB – a **5-stage pipeline** (pipeline of depth 5).
- Assume the time delay through each stage is the same (say 1 clock cycle).

$$\text{Pipeline speedup} = \text{time(non-pipelined)} / \text{time(pipelined)}$$

Pipeline speedup (5-stage, 1 cycle per stage)

- Non-pipelined processor: **5n** cycles for n instructions.
- Pipelined processor: **4 + n** cycles.

$$\text{Speedup} = 5n / (4 + n) \rightarrow 5 \text{ as } n \rightarrow \infty$$

KEY POINT

A pipeline with **p stages** could give a speedup of **p** (vs a non-pipelined processor taking p cycles per instruction), i.e. the program would run p times faster **if an instruction completes on every clock cycle**.

4.2 Pipeline Hazards

DEFINITION

Hazard: a dangerous situation. A **pipeline hazard** is a situation where an instruction **cannot proceed through the pipeline as it should** (from the perspective of correct program execution).

Hazard type	Description
1. Structural hazard	When 2 or more instructions in the pipeline need to use the same resource at the same time
2. Data hazard	When an instruction depends on the data result of a prior instruction that is still in the pipeline
3. Control hazard	A hazard that arises due to control transfer instructions

A. Structural Hazard

- Example: instruction i (LW R3, 8(R2)) is in the **MEM** stage while instruction i+3 is in **IF – MEM** and **IF use memory at the same time**.
- Instruction i+3 must wait (bubbles **B**) and is fetched later.

Solving Structural Hazards

- Design main memory so that it can handle **2 memory requests at the same time – double-ported memory**.
- Or, since memory delays are hidden by cache memories: include a **separate instruction cache** (for the IF stage) and **data cache** (for the MEM stage).
- **Identify the possible structural hazards and design so as to eliminate them.**

B. Data Hazard

- Example: **add R3,R1,R2** (instruction i) updates R3; **sub R4,R3,R8** (instruction i+1) reads R3 – R3 is read by i+1 **before** it is updated by i.
- Idea: **delay (stall)** the progress of instruction i+1 through the pipeline until the data is available in register R3.

Solving Data Hazards

1. **Interlock** – hardware included in the processor to **detect such a data dependency and stall the dependent instruction**.
 2. **Forwarding or Bypassing** – forward the result to EX as soon as it is available anywhere in the pipeline.
 3. **Load delay slot** – build the hardware to assume that an instruction that uses a load value is **separated from the load instruction**.
 4. **Instruction scheduling** – reorder the instructions of the program so that dependent instructions are far enough apart. Done either by the **compiler** before the program runs (**Static Instruction Scheduling**) or by the **hardware** while the program runs (**Dynamic Instruction Scheduling**).
- **Interlock example** (add, sub, or with dependent registers): the dependent instruction shows stall cycles in the timeline until the operand is ready.
 - **Forwarding:** the result is available at the **output of the ALU** (special-purpose register ALUout) now; it is forwarded to the EX stage of the following instructions (e.g. **add R3,R1,R2** → **sub R5,R3,R4** → or **R7,R3,R6**). This needs a **modified processor datapath** with forwarding paths.

KEY POINT

Forwarding is not always possible: for **LW R3,-4(R1)** followed by **SUB R5,R3,R4**, the loaded value is available only after MEM, so a stall (load delay) is still required.

- Notes from ISA manual: for **load instructions** the loaded value might not be available in the destination register for use by the instruction immediately following the load – **LOAD DELAY SLOT**. For **control transfer instructions** the transfer of control takes place only following the instruction immediately after the control transfer instruction – **BRANCH DELAY SLOT**.

Static Instruction Scheduling

- Reorder the instructions of the program to **eliminate data hazards** (or in general reduce execution time).
- Reordering must be **safe** – should not change the meaning of the program.
- Two instructions can be exchanged **if they are independent of each other**.

Program fragment:	Scheduled (reordered):
LW R3, 0(R1) <-1 stall	LW R3, 0(R1)
ADDI R5, R3, 1	LW R13, 0(R11)
ADD R2, R2, R3	ADDI R5, R3, 1
LW R13, 0(R11) <-1 stall	ADD R2, R2, R3
ADD R12, R13, R3	ADD R12, R13, R3

2 stalls

0 stalls

- [Clarification: the slide shows 2 stalls before and 0 stalls after scheduling. The second load is moved up so each load and its dependent instruction are separated by an independent instruction.]

Dynamic Instruction Scheduling

- With dynamic scheduling, instructions go through IF and ID, then wait in the **Instruction Window / Instruction Queue**, then execute on **functional units** (Floating-point adder, FP multiplier, Integer ALU, Integer multiplier, Memory unit), then WB.
- The hardware dynamically schedules instructions from the Instruction Window for execution on the functional units.
- Instructions could execute in an **order different from that specified by the program**, with the same result – such processors are called “**out of order**” processors (as opposed to “in order” processors).

Kinds of Data Dependence

Kind	Example	Meaning
True dependence (RAW)	ADD R1,R2,R3 SUB R4,R1,R5	Second instruction reads R1 produced by first
Anti-dependence (WAR)	ADD R1,R2,R3 SUB R2,R4,R5	Second instruction writes R2 that first reads
Output dependence (WAW)	ADD R1,R2,R3 SUB R1,R4,R5	Both write R1

C. Control Hazard

- Example: **BEQZ R3,out** – condition and target are resolved only in the **EX stage**; meanwhile the processor does not know whether to fetch instruction i+1 or the target (bubbles B).
- **Observation:** since the branch is resolved only in the EX stage, there must be **2 stall cycles after every conditional branch instruction**.

Reducing the Impact of Branch Stall

- A conditional branch involves 2 activities: (1) evaluating the branch **condition** (taken / not taken) and (2) computing the branch **target address**.
- To reduce the stall: evaluate the condition **earlier (ID stage)** and compute the target address **earlier (ID stage)**.
- The number of stall cycles is then reduced to **1 cycle**.

Control Hazard Solutions

1. Static Branch Prediction

- **Prediction** = reasoning about the future / guessing what is going to happen. **Static** = the behaviour of a branch instruction is predicted **once before the program starts executing**.
- **What if the guess is incorrect?** The pipeline hardware must detect the misprediction and take appropriate corrective action.
- **Static Not-Taken policy:** the hardware is built to fetch next from **PC + 4**. After the ID stage, if the branch condition is **false (not taken)**, continue with the fetched instruction – **0 stall cycles**.
- **Else, squash** (cancel/annul the processing of) the fetched instruction and **re-fetch from the branch target address** – **1 stall cycle**.

- Thus, the **average branch penalty is < 1 cycle**.

Branch outcome (static not-taken)	Action	Stall cycles
Condition FALSE (not taken)	Continue with instruction i+1 (from PC+4)	0
Condition TRUE (taken)	Squash instruction i+1; fetch from branch target	1

2. Delayed Branching

- Design hardware so that **control transfer takes place after a few of the following instructions** (e.g. BEQ R1,R2,target followed by ADD R3,R2,R3 – the ADD executes in the **branch delay slot**).
- **Filling delay slots (compiler's role)**: instructions that do not affect the branching condition can be put in the delay slot. Sources:

Source	Useful when
From the branch target address	Only valuable when the branch is taken
From the fall-through (branch not taken path)	Only valuable when the branch is not taken
From before the branch	Useful whether the branch is taken or not

- When filled from branch target or fall-through, **patch-up code may be needed**.
- It may still be beneficial, depending on branching frequency. **The more delay slots, the harder it is to fill them usefully.**
- **If no instruction can be found**: the compiler must insert an instruction that does nothing (e.g. **ADD R0,R0,R0**) – a **No-Operation (NOP)** instruction. It has practically the same effect as a **STALL** cycle.

SECTION SUMMARY

Pipelining gives up to $p \times$ speedup ($5n/(4+n)$ for 5 stages). Hazards: structural (resources), data (RAW/WAR/WAW), control (branches). Fixes: double-ported memory/split caches; interlock, forwarding, load delay slot, scheduling; static prediction, delayed branching with NOPs.

PART 5 – Instruction-Level Parallelism (ILP)

5.1 Introduction

- **Pipelining became a universal technique in 1985.** It overlaps execution of instructions and exploits **Instruction-Level Parallelism**.
- Beyond this, there are two main approaches:

Hardware-based dynamic approaches	Compiler-based static approaches
Used in server and desktop processors. Not used as extensively in PMP (personal mobile device) processors.	Not as successful outside of scientific applications.

Pipeline CPI

Pipeline CPI = Ideal pipeline CPI + Structural stalls + Data hazard stalls + Control stalls

- When exploiting ILP, the goal is to **minimize CPI** [the original note says “maximize”, which is a slip – the equation above is to be minimized].
- Parallelism **within a basic block is limited** – typical size of a basic block = **3–6 instructions**. Must **optimize across branches**.

5.2 Data Dependence & Hazards

- **Loop-level parallelism:** unroll the loop statically or dynamically; use **SIMD** (vector processors and GPUs).
- **Challenge: data dependency.** Instruction j is data dependent on instruction i if:
 - Instruction i produces a result that may be used by instruction j, **or**
 - Instruction j is data dependent on instruction k and instruction k is data dependent on instruction i (chain).
- **Dependent instructions cannot be executed simultaneously.**
- Dependencies are a **property of programs**; the **pipeline organization** determines whether a dependence is detected and whether it causes a stall.

Data dependence conveys

1. The **possibility of a hazard**
 2. The **order** in which results must be calculated
 3. An **upper bound** on exploitable instruction-level parallelism
- Dependencies that flow through **memory locations** are difficult to detect.

Name Dependence

- Two instructions use the **same name but there is no flow of information** – not a true data dependence, but a problem when **reordering instructions**.

Type	Definition	Note
Antidependence	Instruction j writes a register or memory location that instruction i reads	Initial ordering (i before j) must be preserved
Output dependence	Instructions i and j write the same register or memory location	Ordering must be preserved

- To resolve, use **renaming techniques**.

Data Hazards

Hazard	Name	Corresponds to
RAW	Read after write	True dependence
WAW	Write after write	Output dependence
WAR	Write after read	Antidependence

Control Dependence

- Ordering of instruction *i* with respect to a **branch instruction**.
- An instruction **control dependent** on a branch **cannot be moved before** the branch so that its execution is no longer controlled by the branch.
- An instruction **not control dependent** on a branch **cannot be moved after** the branch so that its execution is controlled by the branch.

EXAMPLE

Example 1:

```
DADDU R1,R2,R3
BEQZ  R4,L
DSUBU R1,R1,R6
L:    ...
OR     R7,R1,R8
```

The **OR** instruction is dependent on **DADDU** and **DSUBU**.

EXAMPLE

Example 2: (assume R4 is not used after *skip*)

```
DADDU R1,R2,R3
BEQZ  R12,skip
DSUBU R4,R5,R6
DADDU R5,R4,R9
skip:
OR     R7,R8,R9
```

It is **possible to move DSUBU before the branch**.

5.3 Compiler Techniques for Exposing ILP

Pipeline Scheduling

- **Separate a dependent instruction from the source instruction by the pipeline latency of the source instruction.**

Example loop:

```
for (i=999; i>=0; i=i-1)
    x[i] = x[i] + s;
```

Pipeline Stalls (original loop)

```

Loop:  L.D    F0,0(R1)
       stall
       ADD.D  F4,F0,F2
       stall
       stall
       S.D    F4,0(R1)
       DADDUI R1,R1,#-8
       stall      (assume integer load latency is 1)
       BNE   R1,R2,Loop

```

Pipeline Scheduling (scheduled code)

```

Loop:  L.D    F0,0(R1)
       DADDUI R1,R1,#-8
       ADD.D  F4,F0,F2
       stall
       stall
       S.D    F4,8(R1)
       BNE   R1,R2,Loop

```

Loop Unrolling

DEFINITION

Loop unrolling is a loop transformation technique that helps optimize the execution time of a program – basically **remove or reduce iterations**. It increases the program's speed by **eliminating loop control instructions and loop test instructions**.

Without loop unrolling	With loop unrolling
<pre> for (int i=0; i<5; i++) printf("Hello\n"); // prints Hello 5 times </pre>	<pre> printf("Hello\n"); printf("Hello\n"); printf("Hello\n"); printf("Hello\n"); printf("Hello\n"); </pre>

Advantages	Disadvantages
Increases program efficiency. Reduces loop overhead. If statements in the loop are not dependent on each other, they can be executed in parallel.	Increased program code size, which can be undesirable. Possible increased register usage in a single iteration to store temporary variables, which may reduce performance.

Unroll by a factor of 4 (assume number of elements divisible by 4)

- **Eliminate unnecessary instructions** (drop the repeated DADDUI & BNE).

```

Loop:  L.D    F0,0(R1)
       ADD.D  F4,F0,F2
       S.D    F4,0(R1)           ; drop DADDUI & BNE
       L.D    F6,-8(R1)
       ADD.D  F8,F6,F2
       S.D    F8,-8(R1)         ; drop DADDUI & BNE
       L.D    F10,-16(R1)
       ADD.D  F12,F10,F2
       S.D    F12,-16(R1)       ; drop DADDUI & BNE
       L.D    F14,-24(R1)
       ADD.D  F16,F14,F2

```

```

S.D    F16, -24(R1)
DADDUI R1, R1, #-32
BNE    R1, R2, Loop

```

- ***Note:** the number of **live registers** is larger than in the original loop.

Loop Unrolling + Pipeline Scheduling

```

Loop:  L.D    F0, 0(R1)
        L.D    F6, -8(R1)
        L.D    F10, -16(R1)
        L.D    F14, -24(R1)
        ADD.D  F4, F0, F2
        ADD.D  F8, F6, F2
        ADD.D  F12, F10, F2
        ADD.D  F16, F14, F2
        S.D    F4, 0(R1)
        S.D    F8, -8(R1)
        DADDUI R1, R1, #-32
        S.D    F12, 16(R1)
        S.D    F16, 8(R1)
        BNE    R1, R2, Loop

```

Strip Mining

DEFINITION

Strip-mining (also called **loop sectioning**) is a loop transformation technique for enabling **SIMD encodings** of loops, as well as a means of improving memory performance. A large loop is fragmented into smaller segments or **strips**.

- It **increases temporal and spatial locality** in the data cache if the data are reusable in different passes of an algorithm.
- It **reduces the number of iterations** of the loop by a factor of the length of each vector (number of operations per SIMD operation).
- **Unknown number of loop iterations?** Let number of iterations = **n**; goal: make **k** copies of the loop body. Generate a **pair of loops**: the first executes **$n \bmod k$** times; the second executes **n / k** times.

SECTION SUMMARY

ILP is limited by basic block size (3–6 instrs) and by data, name and control dependences. Compilers expose ILP using pipeline scheduling, loop unrolling and strip mining; unrolling cuts loop overhead but raises code size and register pressure.

PART 6 – Branch Prediction & Dynamic Scheduling

6.1 Branch Prediction

Predictor	Idea
Basic 2-bit predictor	For each branch, predict taken or not taken. If the prediction is wrong two consecutive times, change the prediction.
Correlating predictor (two-level predictor)	Uses the recent behaviour of other branches as well as the branch being predicted.
Tournament predictor	Combines a correlating predictor with a local predictor .

Correlating Predictor

- 2-bit predictor schemes use **only the recent behaviour of a single branch** to predict its future behaviour.
- Accuracy may improve if we also look at the **recent behaviour of other branches**.

```
if (aa==2) aa=0;    // b1
if (bb==2) bb=0;    // b2
if (aa!=bb) {       // b3
    ...
```

- Label these branches **b1, b2, b3**. The behaviour of **b3 is correlated with b1 and b2**.
- If b1 and b2 are both **not taken** (both conditions evaluate to true, so aa and bb are both assigned 0), then aa and bb are clearly equal. [Textbook: b3 is then **not taken**, since aa != bb is false; the slide text says “taken”, which looks like a slip.]
- A predictor using only a single branch’s behaviour can **never capture this behaviour**.
- Branch predictors that use the behaviour of other branches are called **correlating predictors or two-level predictors**.

6.2 Dynamic Scheduling

- Rearrange the order of instructions to reduce stalls while maintaining data flow – done at run time, by hardware.

Advantages	Disadvantages
Compiler doesn’t need knowledge of the microarchitecture . Handles cases where dependencies are unknown at compile time .	Substantial increase in hardware complexity. Complicates exceptions.

- Dynamic scheduling implies **out-of-order execution** and **out-of-order completion**, creating the possibility of **WAR and WAW hazards**.

Tomasulo's Approach

- Tracks when operands are available.**
- Introduces **register renaming in hardware** – minimizes WAW and WAR hazards.

Register Renaming (example)

```
DIV.D F0,F2,F4
ADD.D F6,F0,F8      ; antidependence (F8)
```

```

S.D    F6,0(R1)
SUB.D  F8,F10,F14    ; antidependence
MUL.D  F6,F10,F8     ; + name dependence with F6

```

After renaming (temporary registers **S** and **T**):

```

DIV.D  F0,F2,F4
ADD.D  S,F0,F8
S.D    S,0(R1)
SUB.D  T,F10,F14
MUL.D  F6,F10,T

```

- **Now only RAW hazards remain**, which can be strictly ordered.

Register renaming is provided by Reservation Stations (RS)

- An RS contains: the **instruction**; **buffered operand values** (when available); the **reservation station number** of the instruction providing the operand values.
- RS **fetches and buffers an operand as soon as it becomes available** (not necessarily involving the register file).
- Pending instructions designate the RS to which they will send their output. Result values are **broadcast on a result bus, the Common Data Bus (CDB)**.
- **Only the last output updates the register file.**
- As instructions are issued, the register specifiers are **renamed with the reservation station**.
- There **may be more reservation stations than registers**.

Tomasulo's Algorithm – Structure & Three Steps

- **Load and store buffers** contain data and addresses and act like reservation stations.

Step	What happens
1. Issue	Get next instruction from the FIFO queue . If an RS is available, issue the instruction to the RS with operand values if available. If operand values are not available, stall the instruction.
2. Execute	When an operand becomes available, store it in any reservation stations waiting for it. When all operands are ready , execute the instruction. Loads and stores are maintained in program order through effective address. No instruction is allowed to initiate execution until all branches that precede it in program order have completed.
3. Write result	Write result on the CDB into reservation stations and store buffers. (Stores must wait until address and value are received.)

MNEMONIC

Tomasulo = Issue → Execute → **W**rite result ("I-E-W").

6.3 Hardware-Based Speculation

- **Execute instructions along predicted execution paths but only commit the results if the prediction was correct.**

DEFINITION

Instruction commit: allowing an instruction to update the register file when the instruction is **no longer speculative**.

- Need an additional piece of hardware to **prevent any irrevocable action until an instruction commits** (i.e. updating state or taking an exception).

Reorder Buffer (ROB)

- Holds the **result of an instruction between completion and commit**.

ROB field	Meaning
Instruction type	Branch / store / register operation
Destination field	Register number
Value field	Output value
Ready field	Has it completed execution?

- Reservation stations are modified accordingly.
- **Register values and memory values are not written until an instruction commits.**
- **On misprediction:** speculated entries in the ROB are **cleared**.
- **Exceptions:** not recognized until the instruction is **ready to commit**.

SECTION SUMMARY

Branch predictors: 2-bit → correlating → tournament. Dynamic scheduling (Tomasulo): reservation stations + CDB rename registers, removing WAR/WAW. Speculation adds a Reorder Buffer so results commit in order.

PART 7 – Multiple Issue, VLIW & Advanced Instruction Delivery

7.1 Multiple Issue and Static Scheduling

- To achieve $CPI < 1$, we need to **complete multiple instructions per clock**.
- Solutions:
 - **Statically scheduled superscalar** processors
 - **VLIW** (Very Long Instruction Word) processors
 - **Dynamically scheduled superscalar** processors

VLIW Processors

- **Package multiple operations into one instruction.**
- Example VLIW processor: **one integer instruction (or branch), two independent floating-point operations, two independent memory references.**
- There must be **enough parallelism in the code to fill the available slots.**

VLIW disadvantages
Statically finding parallelism
Code size
No hazard detection hardware
Binary code compatibility

7.2 Dynamic Scheduling, Multiple Issue, and Speculation

- Modern microarchitectures combine **dynamic scheduling + multiple issue + speculation**.
- Two approaches:
 - Assign reservation stations and update the pipeline control table in **half clock cycles** – only supports **2 instructions/clock**.
 - Design logic to handle **any possible dependencies** between the instructions.
 - **Hybrid approaches.**

Multiple Issue (bundle rules)

- **Limit the number of instructions of a given class** that can be issued in a “bundle” (e.g. one FP, one integer, one load, one store).
- **Examine all the dependencies** among the instructions in the bundle.
- If dependencies exist in the bundle, **encode them in reservation stations**.
- Also need **multiple completion/commit**.

Example loop used for multiple-issue analysis

```
Loop: LD      R2,0(R1)      ; R2 = array element
      DADDIU  R2,R2,#1      ; increment R2
      SD      R2,0(R1)      ; store result
      DADDIU  R1,R1,#8      ; increment pointer
      BNE     R2,R3,LOOP    ; branch if not last element
```

(The slides then show this loop executing **without speculation** and **with speculation** on a dual-issue processor.)

7.3 Advanced Techniques for Instruction Delivery and Speculation

Branch-Target Buffer (BTB)

- Need **high instruction bandwidth**. A BTB is a **next-PC prediction buffer**, indexed by current PC.

Branch Folding

- Optimization: **larger branch-target buffer**; add the **target instruction into the buffer** to deal with the longer decoding time required by the larger buffer – “**branch folding**”.

Return Address Predictor

- Most unconditional branches come from **function returns**.
- The same procedure can be called from multiple sites, causing the buffer to potentially forget the return address from previous calls.
- Solution: create a **return address buffer organized as a stack**.

Integrated Instruction Fetch Unit

- Design a **monolithic unit** that performs: **branch prediction**, **instruction prefetch** (fetch ahead), **instruction memory access and buffering** (deal with crossing cache lines).

Register Renaming vs Reorder Buffers

- Instead of virtual registers from reservation stations and ROB, create a **single register pool** containing **visible registers and virtual registers**.
- Use a **hardware-based map** to rename registers during issue; **WAW and WAR hazards are avoided**.
- **Speculation recovery occurs by copying during commit**. Still need a ROB-like queue to update the table in order.
- **Simplifies commit**: record that the mapping between architectural register and physical register is no longer speculative; free up the physical register holding the older value – i.e. **swap physical registers on commit**.
- **Physical register de-allocation is more difficult**.

Integrated Issue and Renaming

1. Issue logic **pre-reserves enough physical registers** for the bundle.
2. Issue logic **finds dependencies within the bundle** and maps registers as necessary.
3. Issue logic **finds dependencies between the current bundle and already in-flight bundles** and maps registers as necessary.

How Much to Speculate?

- **Mis-speculation degrades performance and power** relative to no speculation; it may cause additional misses (cache, TLB).
- Prevent speculative code from causing **higher-cost misses** (e.g. L2).
- **Speculating through multiple branches** complicates speculation recovery; **no processor can resolve multiple branches per cycle**.

Energy Efficiency

- Speculation is **only energy efficient when it significantly improves performance**.
- **Value prediction** – uses: loads that load from a constant pool; instructions that produce a value from a small set of values. **Not incorporated into modern processors**.
- A similar idea – **address aliasing prediction** – is used (the slide text ends here).

SECTION SUMMARY

CPI < 1 needs multiple issue (superscalar or VLIW). Fetch is supported by BTB, branch folding, return-address stack and an integrated fetch unit. Speculation costs energy unless it significantly improves performance.

QUICK REVISION – Formula Sheet

Topic	Formula
Dynamic energy	$\frac{1}{2} \times C \times V^2$
Dynamic power	$\frac{1}{2} \times C \times V^2 \times f(\text{switched})$
Static power	$I(\text{static}) \times V$
Die yield (Bose–Einstein)	$\text{Wafer yield} \times 1/(1 + \text{defects/area} \times \text{die area})^N$
Cost of die	$\text{Cost of wafer} / (\text{Dies per wafer} \times \text{Die yield})$
Dies per wafer	$\pi(d/2)^2/A - \pi d/v(2A)$
MTBF	$\text{MTTF} + \text{MTTR}$
Availability	$\text{MTTF} / \text{MTBF}$
Speedup	$\text{Exec time}(Y) / \text{Exec time}(X)$
Amdahl's Law	$1 / [(1 - F) + F/S]$
Amdahl (N workers)	$S = N / (B \times N + (1 - B))$
CPU time	$IC \times CPI \times \text{Clock cycle time}$
CPU clock cycles	$\sum IC_i \times CPI_i$
Pipeline speedup (5-stage)	$5n / (4 + n) \rightarrow 5$
Pipeline CPI	Ideal CPI + Structural + Data hazard + Control stalls

Quick Comparison Tables

	Static scheduling	Dynamic scheduling
Done by	Compiler (before run)	Hardware (at run time)
Needs microarchitecture knowledge	Yes	No
Unknown dependencies at compile time	Cannot handle	Handles them
Hardware complexity	Low	Substantially higher

Hazard fix	Hazard type	Idea
Double-ported memory / split I-D cache	Structural	Allow 2 memory accesses at once
Interlock	Data	Detect and stall
Forwarding / bypassing	Data	Send result early to EX
Load delay slot / scheduling	Data	Separate dependent instructions
Static not-taken / delayed branch	Control	Predict or fill slot with useful work/NOP
Register renaming (Tomasulo)	WAR / WAW	Remove name dependences